

ISO SECURE

Plataforma de Cumplimiento ISO 27001

Etapa 4: Pruebas y Despliegue

Actividad Formativa 4 - 5%

Agustín Peralta — Universidad de San Buenaventura

Entrega: 18 de Mayo de 2026

Resumen

Este documento presenta las pruebas de seguridad dinámicas (DAST) y la configuración segura aplicadas en el proyecto ISO SECURE durante la fase de pruebas y despliegue. Cubre la estrategia DAST con OWASP ZAP, Nuclei y pruebas de fuzzing; el hardening de infraestructura (Nginx, Docker, PostgreSQL, sistema operativo); las cabeceras de seguridad HTTP; la gestión de secretos en producción; la estrategia de backups y los procedimientos de respuesta a incidentes. Las prácticas se alinean con OWASP ASVS 4.0, CIS Benchmarks y los controles A.8 (controles tecnológicos) y A.5 (organizacionales) de ISO/IEC 27001:2022.

Índice

1. Introducción	2
2. Análisis Dinámico (DAST)	2
2.1. Herramientas seleccionadas	2
2.2. Estrategia DAST por capas	2
2.2.1. Capa 1 — Reconocimiento (sin login)	2
2.2.2. Capa 2 — Autenticación	2
2.2.3. Capa 3 — Autorizado (con cuenta de prueba)	3
2.2.4. Capa 4 — Fuzzing	3
2.3. Plan de ejecución	3
2.4. Catálogo esperado de hallazgos y mitigaciones	3
3. Configuración Segura — Hardening por capa	4
3.1. Sistema Operativo (host)	4
3.2. Docker / Contenedores	5
3.3. Nginx (reverse proxy + TLS termination)	5
3.4. PostgreSQL (Supabase managed)	6
3.5. Aplicación FastAPI	6
3.6. Frontend React (build estático)	7

4. Gestión de Secretos en Producción	7
4.1. Principios	7
4.2. Inventario de secretos	7
5. Estrategia de Despliegue	7
5.1. Pipeline de promoción	7
5.2. Estrategia blue-green simplificada (Coolify)	8
5.3. Plan de rollback	8
6. Backups y Continuidad	8
7. Respuesta a Incidentes	8
7.1. Niveles de severidad	8
7.2. Procedimiento IR (control A.5.24-A.5.28)	8
8. Monitoreo y Observabilidad	9
9. Mapeo a ISO 27001:2022	9
10. Conclusiones	10

1. Introducción

La Etapa 4 cierra el ciclo de construcción con dos focos complementarios:

1. **Análisis Dinámico de Seguridad (DAST):** ejecutar la aplicación y atacarla deliberadamente para descubrir vulnerabilidades que el análisis estático no puede detectar (problemas de configuración, lógica de negocio, sesiones, autenticación).
2. **Configuración Segura:** endurecer cada capa del stack (sistema operativo, contenedores, servidor web, base de datos, aplicación) siguiendo CIS Benchmarks y guías oficiales del proveedor.

A diferencia del SAST (Etapa 3), que analiza código fuente, el DAST analiza el sistema corriendo. Ambos son complementarios y obligatorios para una cobertura de seguridad madura (DevSecOps).

2. Análisis Dinámico (DAST)

2.1. Herramientas seleccionadas

Herramienta	Propósito	Tipo
OWASP ZAP	Scan automatizado + proxy interceptor + spider	DAST general
Nuclei	Plantillas YAML para CVEs conocidas y mis-configs	DAST plantillas
sqlmap	Pruebas dirigidas de inyección SQL	Especializado
nikto	Server scan: archivos comunes, headers, versiones	Web server
testssl.sh	Auditoría de TLS (ciphers, versiones, certs)	TLS/SSL
ffuf	Fuzzing de endpoints y parámetros	Fuzzing
JWT_Tool	Pruebas específicas contra tokens JWT	API/Auth
Burp Suite Community	Análisis manual asistido	Manual

2.2. Estrategia DAST por capas

2.2.1. Capa 1 — Reconocimiento (sin login)

- Spider de ZAP sobre la URL pública: descubrir endpoints expuestos.
- nikto: archivos sensibles (`.git`, `.env`, `backup.sql`).
- testssl.sh: validar TLS 1.3, sin cifrados débiles, HSTS.

2.2.2. Capa 2 — Autenticación

- Enumeración de usuarios vía login (mensajes diferenciados).
- Brute force con throttle/lockout.
- Resistencia a credential stuffing (Have I Been Pwned).
- Validación de JWT: firma, alg=none, kid traversal, exp, nbf.

2.2.3. Capa 3 — Autorizado (con cuenta de prueba)

- ZAP Active Scan con contexto autenticado (script de re-auth).
- Tests de IDOR: cambiar IDs en URL/body.
- Tests multi-tenant: con token de empresa A intentar leer/escribir en empresa B.
- Privilege escalation: con rol supervisor intentar acciones de admin.

2.2.4. Capa 4 — Fuzzing

- ffuf sobre parámetros JSON con payloads de OWASP juice-shop.
- Pruebas de límites: strings de 1MB, enteros negativos, fechas inválidas.
- Inyecciones: SQL, NoSQL, comandos, plantilla (SSTI), LDAP, XPath.

2.3. Plan de ejecución

```

1  # 1. Scan baseline ZAP en CI (cada PR)
2  docker run --rm -v $(pwd):/zap/wrk owasp/zap2docker-stable \
3    zap-baseline.py -t https://staging.iso-secure.app -r zap-
   report.html
4
5  # 2. Nuclei contra staging diariamente
6  nuclei -u https://staging.iso-secure.app \
7    -t cves/ -t misconfiguration/ -t exposures/ \
8    -severity critical,high -o nuclei-report.txt
9
10 # 3. testssl.sh contra produccion (semanal)
11 testssl.sh --severity HIGH https://iso-secure.app
12
13 # 4. ZAP autenticado (release)
14 zap-full-scan.py -t https://staging.iso-secure.app \
15   -z "-config replacer.full_list(0).description=auth \
16       -config replacer.full_list(0).enabled=true \
17       -config replacer.full_list(0).matchtype=REQ_HEADER \
18       -config replacer.full_list(0).matchstr=Authorization \
19       -config replacer.full_list(0).replacement=Bearer $TOKEN
   "

```

2.4. Catálogo esperado de hallazgos y mitigaciones

ID	Hallazgo	Sev.	Mitigación
D-01	Cookies sin flag HttpOnly/Secure	Alta	Configurar atributos cookie + SameSite=Lax
D-02	CORS demasiado permisivo (*)	Alta	Whitelist explícita por entorno

ID	Hallazgo	Sev.	Mitigación
D-03	Headers de seguridad faltantes (CSP, X-Frame)	Media	Añadir en Nginx (sección Hardening)
D-04	Mensajes de error con stack trace en prod	Media	DEBUG=False + handler genérico
D-05	Robots.txt expone rutas internas	Baja	Eliminar referencias internas
D-06	Server: nginx/1.x revela versión	Baja	server_tokens off;
D-07	TLS 1.0/1.1 habilitado	Alta	Solo TLS 1.2+ (ideal 1.3)
D-08	Cifrados RC4/3DES soportados	Alta	Suite de cifrados modernos (Mozilla intermediate)
D-09	Brute force sin lockout	Alta	Rate limit + lockout temporal en Supabase
D-10	JWT alg=none aceptado	Crítica	Validar contra Supabase /auth/v1/user (ya implementado)
D-11	IDOR en /incidents/{id}	Crítica	Verificar ownership en service layer
D-12	Cross-tenant via parámetro	Crítica	Helper _empresa_filter (ya implementado)
D-13	SSRF en webhooks n8n	Alta	Allowlist de hosts salientes
D-14	Open redirect en /login?return_to	Media	Validar dominio destino
D-15	Falta de rate limit global	Alta	slowapi + Nginx limit_req

3. Configuración Segura — Hardening por capa

3.1. Sistema Operativo (host)

- Distribución mínima (Debian slim o Alpine).
- Actualizaciones automáticas de seguridad (unattended-upgrades).
- Usuario no-root para correr servicios.
- SSH: solo claves, sin password, puerto no estándar, fail2ban.
- Firewall ufw: permitir 22, 80, 443; denegar resto.
- Auditd habilitado para eventos sensibles.
- Sin servicios innecesarios; systemd-analyze review.

3.2. Docker / Contenedores

- Imágenes base oficiales y versionadas (no latest).
- Multi-stage builds para reducir superficie de ataque.
- Usuario no-root en el Dockerfile (USER 10001).
- Capabilities mínimas (-cap-drop=ALL).
- Read-only filesystem cuando sea posible.
- Healthchecks declarados.
- Escaneo con Trivy antes de promover a producción.

Ejemplo Dockerfile hardened:

```
1 FROM python:3.13-slim AS builder
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --user --no-cache-dir -r requirements.txt
5
6 FROM python:3.13-slim
7 RUN groupadd -r app && useradd -r -g app -u 10001 app
8 COPY --from=builder /root/.local /home/app/.local
9 COPY --chown=app:app . /app
10 WORKDIR /app
11 USER app
12 ENV PATH=/home/app/.local/bin:$PATH
13 HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import
    urllib.request; urllib.request.urlopen('http://localhost
    :8000/health')"
14 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "
    8000"]
```

3.3. Nginx (reverse proxy + TLS termination)

- TLS 1.2+ obligatorio; 1.3 preferido.
- Suite de cifrados Mozilla intermediate.
- HSTS con max-age=31536000; includeSubDomains; preload.
- server_tokens off; para ocultar versión.
- Rate limiting por IP en endpoints sensibles (login).
- Buffer sizes para mitigar Slowloris.
- Logs en formato JSON con IP, user-agent, status, latencia.

Headers de seguridad obligatorios:

```
1 # nginx headers (extracto)
2 add_header Strict-Transport-Security "max-age=31536000;
   includeSubDomains; preload" always;
3 add_header X-Content-Type-Options "nosniff" always;
4 add_header X-Frame-Options "DENY" always;
5 add_header Referrer-Policy "strict-origin-when-cross-origin"
   always;
6 add_header Permissions-Policy "geolocation=(), microphone=(),
   camera=()" always;
7 add_header Content-Security-Policy "default-src 'self'; script
   -src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-
   inline'; img-src 'self' data: https:; connect-src 'self'
   https://*.supabase.co; frame-ancestors 'none';" always;
```

3.4. PostgreSQL (Supabase managed)

Aunque Supabase gestiona muchos controles, las prácticas que aplicamos del lado cliente son:

- Solo conexiones SSL (sslmode=require).
- Roles separados: `app_user` (CRUD) y `migrator` (DDL).
- Sin `SUPERUSER` para conexiones de aplicación.
- Row Level Security (RLS) habilitado en tablas multi-tenant.
- Backups automáticos diarios + Point-in-Time Recovery 7 días.
- Encriptación at-rest activa por defecto.
- Audit logging habilitado para DDL y privilegios.

3.5. Aplicación FastAPI

- `DEBUG=False` y `docs_url=None` en producción (o tras auth admin).
- Workers Uvicorn $\geq 2 \times$ vCPU para tolerancia.
- Timeouts: `-timeout-keep-alive=5`, `-limit-concurrency=1000`.
- Middleware de logging estructurado con `request_id`.
- Trusted Hosts middleware con whitelist explícita.
- GZip middleware con `minimum_size` para evitar BREACH.

3.6. Frontend React (build estático)

- Source maps solo en staging; deshabilitados en prod.
- Subresource Integrity (SRI) en scripts externos.
- Variables sólo VITE_ (públicas por diseño); nada sensible en bundle.
- Sentry/Posthog desactivados en preview branches.

4. Gestión de Secretos en Producción

4.1. Principios

1. **Nunca en el código.** Detección con `detect-secrets` en pre-commit y CI.
2. **Nunca en logs.** Filtros de redacción en el logger.
3. **Rotación periódica.** 90 días para credenciales operativas.
4. **Mínimo privilegio.** Cada servicio con sus propias credenciales.
5. **Cifrado en reposo.** Vault del proveedor (Supabase Vault, GitHub Secrets, Coolify Secrets).

4.2. Inventario de secretos

Secreto	Uso	Rotación
DATABASE_URL	Conexión PostgreSQL	90 días
SUPABASE_URL	Endpoint del proveedor	estática
SUPABASE_ANON_KEY	Auth cliente (pública por diseño)	180 días
SUPABASE_SERVICE_ROLE_KEY	Operaciones server-side	90 días
JWT_VALIDATION_URL	Endpoint de validación	estática
N8N_WEBHOOK_SECRET	HMAC de webhooks salientes	90 días
SMTP_PASSWORD	Emails transaccionales	90 días

5. Estrategia de Despliegue

5.1. Pipeline de promoción

Entorno	Trigger	Pruebas	Aprobación
PR preview	abrir PR	smoke + ZAP baseline	automática
staging	merge a <code>develop</code>	full ZAP + e2e Playwright	automática
producción	merge a <code>main</code>	smoke post-deploy	manual (1 reviewer)

5.2. Estrategia blue-green simplificada (Coolify)

- Coolify levanta nueva versión del contenedor.
- Healthcheck `/health` debe pasar.
- Si pasa, swap de tráfico en el reverse proxy interno.
- Si falla, rollback automático a la versión previa (último commit verde).

5.3. Plan de rollback

1. Detectar regresión vía healthcheck, monitoreo o reporte de usuario.
2. Coolify: re-deploy del último commit etiquetado como `stable`.
3. Si involucra migración de DB: ejecutar `alembic downgrade -1`.
4. Post-mortem y RCA dentro de 48 hrs (control A.5.27).

6. Backups y Continuidad

Activo	Estrategia	RPO / RTO
Base de datos	PITR Supabase 7 días + snapshot diario externo	1h / 2h
Código	Git remoto (GitHub) + mirror en repo URPE	0 / 30min
Configuración (Coolify)	Export semanal del project a S3 cifrado	24h / 1h
Secretos	Backup cifrado del vault en cold storage	24h / 4h
Logs	Retención 90d + agregación off-site	1h / N/A

7. Respuesta a Incidentes

7.1. Niveles de severidad

Nivel	Descripción	SLA respuesta
P0 (Crítico)	Brecha activa, indisponibilidad total, fuga de datos	15 min
P1 (Alto)	Vulnerabilidad explotable confirmada, degradación severa	1 h
P2 (Medio)	Vulnerabilidad sin exploit público, función no crítica caída	4 h
P3 (Bajo)	Hardening pendiente, hallazgo informativo	5 días hábiles

7.2. Procedimiento IR (control A.5.24-A.5.28)

1. **Detección** → alerta automática o reporte manual.
2. **Triage** → asignar severidad y propietario.

3. **Contención** → aislar (revocar tokens, bloquear IP, deshabilitar usuario, blue-green rollback).
4. **Erradicación** → corregir causa raíz, parchear, actualizar.
5. **Recuperación** → restaurar servicio + validar integridad.
6. **Lecciones aprendidas** → post-mortem + actualización de catálogo de amenazas (Etapa 2).

8. Monitoreo y Observabilidad

- **Logs:** salida JSON estructurada → agregador (Loki/Grafana o Better Stack).
- **Métricas:** CPU, RAM, latencia p95, error rate (Prometheus + Grafana).
- **Traces:** OpenTelemetry para request flow end-to-end.
- **Alertas:** error rate > 1 % por 5min, latencia p95 > 2s, healthcheck fallido.
- **Dashboard ejecutivo:** disponibilidad mensual (SLA 99.5 %), MTTR, número de incidentes por severidad.

9. Mapeo a ISO 27001:2022

Control Anexo A	Implementación
A.5.24 Planificación de IR	Procedimiento documentado + SLAs por severidad
A.5.25 Evaluación de eventos	Triaje formal en cada alerta
A.5.27 Lecciones aprendidas	Post-mortem en 48h + retroalimentación al modelo de amenazas
A.5.30 Continuidad TIC	RPO/RTO definidos + PITR + mirror de código
A.8.7 Protección contra malware	Trivy en CI + actualizaciones de base OS
A.8.8 Gestión de vulnerabilidades técnicas	DAST + Nuclei + Dependabot
A.8.9 Gestión de la configuración	Coolify + IaC + secretos en vault
A.8.13 Backup de información	Backups diarios + PITR
A.8.15 Logging	Logs JSON estructurados + retención 90d
A.8.16 Monitoreo de actividades	Prometheus + alertas
A.8.20 Seguridad de redes	TLS 1.3 + firewall + rate limit
A.8.23 Filtrado web	CSP estricto + CORS allowlist
A.8.26 Requisitos de seguridad de aplicaciones	DAST regular como gate de release
A.8.27 Arquitectura segura	Hardening por capa documentado
A.8.28 Codificación segura	Resultados DAST retroalimentan estándares de código

10. Conclusiones

La Etapa 4 cierra el ciclo de aseguramiento técnico previo a la entrega final:

1. **DAST automatizado** en CI (ZAP baseline en cada PR; Nuclei diario; full ZAP autenticado por release).
2. **Hardening por capa** documentado: SO, Docker, Nginx, PostgreSQL, FastAPI, frontend.
3. **Gestión de secretos** con vault, rotación 90 días y detección automática en pre-commit/CI.
4. **Despliegue blue-green** mediante Coolify con rollback automático.
5. **Backups y continuidad** con RPO 1h / RTO 2h en la base de datos.
6. **Respuesta a incidentes** con SLA por severidad y post-mortem obligatorio.

Las pruebas dinámicas no son un evento puntual: son un proceso continuo que retroalimenta al modelo de amenazas (Etapa 2) y a las prácticas de codificación segura (Etapa 3), cumpliendo el ciclo PDCA exigido por ISO 27001:2022.

Referencias

- OWASP Web Security Testing Guide (WSTG) v4.2.
- OWASP Application Security Verification Standard (ASVS) 4.0.3.
- CIS Docker Benchmark v1.6 y CIS Nginx Benchmark v2.0.
- Mozilla SSL Configuration Generator (Intermediate Profile).
- NIST SP 800-61 Rev. 2 — Computer Security Incident Handling Guide.
- ISO/IEC 27001:2022 — Anexo A.5 y A.8.